

第 13 讲：Context Engineering 基础：选择、压缩、检索与记忆边界

Coding Agent Notes (June 2026)

本讲在系列中的位置：#11 讲了 product workflow 里的 context 控制面，#12 讲了多 agent 的 context isolation。

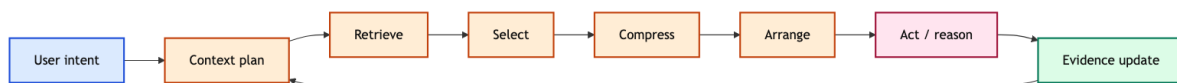
已知前提：读者已经知道 coding agent 会反复读文件、调用工具、运行测试、保存 evidence。

本讲新增：context engineering 的基础词汇：selection、compression、retrieval、memory、budget、grounding、context rot。

读完后应该能回答：为什么更长 context window 不等于更好 agent；怎样组织 AGENTS.md / CLAUDE.md；什么时候用 retrieval、memory、summary、tool，而不是继续往 prompt 里塞文字。

1 核心命题

context engineering 是 coding agent 的信息控制层。它决定在每一轮推理前，哪些信息应该进入模型上下文，哪些信息应该留在文件系统、日志、检索索引、记忆或工具里，哪些信息应该被压缩，哪些信息应该原样保留。



这个 pipeline 可以读成一个循环：用户意图进入系统后，agent 先形成 context plan；再通过搜索、文件读取、日志、测试、外部文档或记忆检索获得候选信息；接着选择高价值内容、压缩低价值内容、安排顺序；模型基于这些上下文行动；新的 evidence 又反过来更新下一轮 context plan。

所以，context engineering 不是“把上下文窗口塞满”。它更像数据库查询优化、cache 管理和工程手册维护的混合体：目标是在有限注意力预算里放入最能改变下一步决策的信息。

Insight. 一个 coding agent 的失败，经常不是模型完全不会写代码，而是它在关键一轮看错了上下文：缺了测试命令、用了过时规则、读了无关文件、把猜测当 evidence，或者在太长的上下文里丢掉真正关键的信号。

2 Context vs Prompt

prompt engineering 主要关心任务怎样表达：目标、语气、格式、few-shot 示例、禁止事项。它仍然重要，但对长期 coding agent 来说不够。因为 agent 的输入不是一段静态 prompt，而是一个不断变化的信息状态：系统指令、项目规则、当前任务、文件片段、工具定义、运行输出、对话历史、memory、检索结果、diff 和失败日志。

Anthropic 的 context engineering 文章把它定义为在 inference 时 curate 和 maintain 最优 token set 的策略。这个定义的关键不是“token set”，而是“maintain”：agent 每走一步都会产生新信息，下一步上下文需要重新组织。

层次	关注点	典型问题
Prompt	怎样说任务	输出格式、风格、few-shot
Context	给模型看什么	文件、日志、规则、证据
Policy	系统允许什么	权限、sandbox、hooks、review
Memory	长期保留什么	偏好、项目经验、调试结论

Context 不是越多越好。

Prompt 可以短而明确；context 可以多源而动态。但二者都必须服务于下一步决策。不能影响决策的信息，就是噪声；不能被验证的信息，就是风险；已经过时的信息，就是误导。

3 Context 是有限注意力预算

长 context window 解决的是容量问题，不自动解决注意力分配问题。Lost in the Middle 研究显示，在多文档问答和 key-value retrieval 中，相关信息出现在输入开头或结尾时效果往往更好，而出现在中间时性能可能明显下降。这不是说所有现代模型都会同样失败，而是提醒我们：能放进去，不等于能稳定用起来。

这对 coding agent 特别重要。一个 repo 里有 README、测试、配置、旧日志、issue、PR comments、用户聊天、generated diff、tool outputs。把它们全部塞进上下文，会让模型看起来“知道很多”，但真正需要的失败栈、函数签名或测试命令可能被埋在中间。

误区	为什么危险	更好的做法
全量塞入	关键证据被噪声稀释	先检索再选择
只看最近消息	忘记项目规则和旧失败	保留 durable rules
只做 summary	丢掉精确日志和 diff	原文证据保留引用
盲信 memory	过时经验误导当前任务	memory 可审计可更新

因此，context engineering 的目标是 high-signal context，而不是 maximum context。对 coding agent 来说，high signal 通常包括：当前失败日志、相关文件片段、测试命令、接口约束、项目规范、最近 diff、明确用户要求和可复现证据。

3.1 Context Budget Accounting

可以把上下文窗口看成一个预算表，而不是一个垃圾桶。每放入一类信息，就要问它是否比其他候选信息更能改善下一步决策。

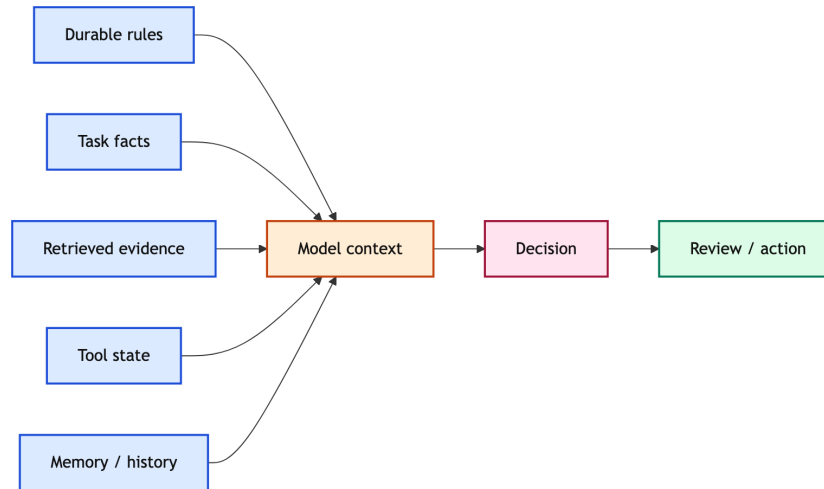
Useful Context = Rules + Task Facts + Evidence + Tool State + Memory – Noise – Stale Assumptions.

这个式子提醒我们：上下文的价值不是 token 数，而是 signal-to-noise ratio。一个 300 token 的失败日志可能比 10,000 token 的 README 更重要；一个明确的“do not edit generated files”规则可能比一段长风格说明更重要；一个 return code 和命令行可能比 agent 对失败原因的总结更重要。

预算项	典型内容	预算原则
Rules	AGENTS.md、CLAUDE.md、style guide	短、稳定、可执行
Facts	本轮目标、范围、验收标准	越靠近任务越高优先级
Evidence	日志、测试、diff、trace	原始证据优先于总结
Tool state	cwd、branch、环境、return code	必须精确，不要猜
Memory	偏好、长期经验、旧结论	需要日期和可编辑性

Context budget accounting 的实用方法，是让 agent 在行动前先列出“本轮需要的三类证据”。例如修 bug 时需要复现证据、相关代码、验证命令；写设计时需要现有架构、约束、候选方案；做 review 时需要 diff、测试覆盖、风险边界。先列预算，再取上下文，能避免无目标搜索。

4 Context Layers



可以把 coding agent 的上下文分成五层。第一层是 durable rules，例如 AGENTS.md、CLAUDE.md、代码规范、构建命令和团队约定。第二层是 task facts，例如 issue 描述、用户本轮目标、验收标准、禁止改动范围。第三层是 retrieved evidence，例如 grep 结果、文件片段、日志、测试输出和文档片段。第四层是 tool/runtime state，例如 cwd、branch、环境变量、依赖安装状态、命令返回码。第五层是 memory/history，例如之前的修正、偏好、长期调试经验和会话摘要。

这些层不应该同权。Durable rules 要稳定但短；task facts 要贴近当前任务；retrieved evidence 要可追溯；tool state 要精确；memory 要可审计。把它们混成一段长 prompt，会导致模型不知道什么是规则、什么是证据、什么只是历史猜测。

Context layer	应该保留什么	常见错误
Durable rules	构建、测试、风格、目录约定	写成抽象口号
Task facts	目标、范围、验收、限制	目标模糊或频繁漂移
Evidence	日志、diff、文件、测试输出	summary 替代原始证据
Tool state	cwd、branch、return code	忽略环境差异
Memory	高频偏好、长期经验	保存一次性猜测

4.1 上下文顺序也是设计

上下文不是一个无序 bag。模型看到的信息顺序会影响它怎样建立任务框架。通常应把稳定规则放在前面，把当前任务和硬约束放在靠近决策的位置，把原始证据放在明确标注的块里，把长历史和低置信度内容压缩到后面或按需加载。

位置	适合放什么	不适合放什么
开头	角色、任务、硬约束、输出格式	长日志和旧对话
中段	相关文件片段、接口、测试命令	大量候选无关文件
证据块	原始错误、diff、return code	未标注的猜测
末尾	下一步计划、问题、待确认项	重要但容易被忽略的约束

这也是为什么 context file 要短。一个很长的 AGENTS.md 可能把真正重要的测试命令埋在中间；一个短而结构化的文件则会让 agent 更容易建立正确默认行为。对于复杂任务，可以用“主文件短、细节链接到 docs”的方式，把 durable rules 和 detailed references 分开。

5 AGENTS.md / CLAUDE.md 是软件 artifacts

AGENTS.md 和 CLAUDE.md 不只是“给 AI 的提示词”。它们是 repo 里的 context artifacts，和 README、CONTRIBUTING、CI 配置一样，需要维护、版本管理和 review。OpenAI Codex 文档建议把如何运行项目、测试、风格、团队实践写进 AGENTS.md，并强调短而准确比长而模糊更有用。Claude Code memory 文档也把 CLAUDE.md 定位为用户写的 persistent instructions。

MSR 2026 的 Context Engineering for AI Agents in Open-Source Software 研究分析了 466 个开源项目里的 AI context files，发现还没有稳定统一的内容结构，并且存在 descriptive、prescriptive、prohibitive、explanatory、conditional 等不同写法。这说明 context files 正处于从个人习惯变成工程规范的早期阶段。

内容类型	好例子	坏例子
Build	‘npm test’ before PR	“确保质量”
Style	prefer small pure functions	“代码要优雅”
Scope	do not edit generated files	“小心点”
Workflow	run formatter after edits	“遵守流程”
Troubleshooting	if DB tests fail, start service X	保存过时 debug 猜测

Context file 要像操作手册，不要像愿望清单。

好的 context file 应该回答 agent 反复需要问的问题：项目怎么跑，测试怎么跑，哪些目录不能改，代码风格有什么硬约束，常见失败怎么诊断。抽象价值观很少帮助 agent 做下一步决策。

5.1 Context File 的生命周期

把 AGENTS.md 或 CLAUDE.md 当成软件 artifact，就意味着它也有生命周期：创建、使用、修改、review、废弃。最常见的问题是只创建不维护。项目换了测试命令、迁移了包管理器、改了目录结构，但 context file 还在告诉 agent 运行旧命令。

阶段	应该做什么	验收信号
Create	写最小可用规则和命令	agent 能跑通基本检查
Use	观察 agent 反复犯的错	错误能映射到缺失规则
Update	把稳定修正写回文件	PR 或 commit 可 review
Audit	删除过时规则和一次性结论	文件变短而更准确
Retire	废弃无效段落或拆到子目录	scope 更清楚

一个好的维护习惯是：每次 agent 因为缺少项目知识而犯错，只在确认这个知识稳定且会复用时才写入 context file。一次性调试发现应该进入 issue comment 或 session summary；长期规则才进入 AGENTS.md / CLAUDE.md。这样 context file 不会变成旧事故堆积。

5.2 一个 Context File 的 Before / After

很多项目的初版 AI context file 会写成这样：‘请写高质量代码，注意测试，遵守项目规范，不要破坏功能。’这句话听起来正确，但几乎不给 agent 任何可执行信息。它没有说明怎样运行测试，规范在哪里，什么叫破坏功能，哪些路径不能改，失败时先检查什么。

更有用的版本应该接近下面这种结构：

Build: backend 用 ‘pytest tests/api -q’；frontend 用 ‘pnpm test’；改数据库逻辑后必须跑 migration smoke test。

Scope: 不要编辑 ‘generated/’、‘vendor/’、‘snapshots/’；API contract 改动必须同步 ‘docs/api.md’。

Style: Python 保持小函数；错误处理返回 typed error；不要吞掉 exception。

Troubleshooting: 如果 integration test 连接失败，先确认本地 Redis 和 Postgres 是否已启动。

Review: 每次 patch summary 必须列出测试命令、未跑测试和潜在风险。

这个例子不是要求所有项目都用同样段落，而是说明 context file 应该把隐性团队知识转成 agent 可执行的决策规则。它应该帮助 agent 少问重复问题、少走错路径、少编造项目事实。

写法	对 agent 的影响	是否推荐
“注意质量”	太抽象，无法执行	否
“运行测试”	不知道运行哪个命令	不够
“运行 ‘pytest tests/api -q’”	可直接执行和验证	推荐
“不要乱改”	没有边界	否
“不要编辑 ‘generated/’”	有明确路径边界	推荐

好的 context file 也应该有 scope。Repo 根目录里的规则应该是全局的；子目录里的规则应该只描述局部差异。比如 backend 和 frontend 有不同测试命令，就应该在对应目录维护更具体的文件，而不是让一个全局文件堆满互相冲突的规则。

6 Retrieval 和 Just-in-Time Context

RAG 的核心思想，是把 parametric memory 和 non-parametric retrieved memory 结合起来。模型参数里有通用知识，外部索引里有可替换、可更新的知识。对于 coding agent，这个思想要进一步扩展：repo 文件、日志、测试输出、commit history、issue、docs、MCP resources 都可以是非参数化上下文。但 coding agent 不应该把所有检索结果一次性塞进去。Anthropic 的文章强调 just-in-time context：保留 file paths、stored queries、web links 等轻量引用，等到需要时再通过工具加载。Claude Code common workflows 也体现这种模式：先问 overview，再缩小到相关组件，再 trace 具体 execution flow。

策略	适合什么	风险
Preload	小 repo、短任务、固定规则	噪声变多
RAG search	文档、issue、历史知识	召回错或排序错
Tool navigation	文件系统、日志、测试	工具输出太长
Just-in-time	大 repo、长任务、数据分析	agent 需要会问好问题

Just-in-time retrieval 的设计重点是让 agent 带着假设去取证，而不是先取一切。一个好的 prompt 不是“读完整个 repo”，而是“先找鉴权入口，列出候选文件，说明每个候选为什么相关，再读取最可能的两个文件”。

6.1 例子：定位登录失败

假设用户说：“最近登录失败率升高，帮我看一下。”一个 context-poor agent 可能直接搜索 ‘login’，读几个文件，然后猜测 password validation 有问题。更好的 context plan 会先把任务拆成三个问题：失败在哪里发生，哪些证据说明它是新问题，修复后怎样验证。

阶段	需要的上下文	为什么
入口定位	routes、controller、auth service、frontend call	找到真实执行路径
失败证据	error logs、metrics、recent deploy、test failures	区分 bug、配置和环境
约束规则	auth policy、security docs、rate limit rules	避免修坏安全边界
候选验证	reproduction test、manual steps、CI command	确认修复不是猜测

这个例子说明，retrieval 不是一次搜索，而是一连串带目标的取证。第一轮可能只需要文件路径和调用图；第二轮才读取具体函数；第三轮读取日志和测试；第四轮再把关键证据压缩成 working summary。每一轮都应该减少不确定性，而不是简单增加 token。

如果上下文计划做得好，最终 prompt 里不需要整个 repo。它只需要：用户目标、当前假设、相关文件片段、失败日志、项目约束、可运行测试和下一步 action。这样的 context 更小，但决策质量更高。

7 Context 和 Tools 的边界

一个常见设计错误，是把工具能随时查询的信息提前塞进上下文。比如文件系统里有完整源码，就没有必要把所有文件内容都放进 prompt；测试可以重新运行，就不应该只保存一个过期 summary；Git 能显示 diff，就不应该依赖 agent 记忆“自己改了什么”。

更好的原则是：稳定规则进入 context，动态状态留给 tools；关键 evidence 进入 context，完整原文留在可追溯位置；高频短信息进入 memory，低频长信息按需检索。

信息类型	放进 context	留给 tool / storage
项目规则	测试命令、禁止范围、风格硬约束	详细文档全文
源码	当前相关函数和接口	整个 repo 文件树
日志	失败摘要和关键原文行	完整日志文件
Git 状态	当前 diff summary、branch	完整 history 和 blame
外部文档	当前决策需要的段落	全文检索索引

这个边界能减少 context rot。工具是可重新查询的，context 是当前推理的一次性工作集。把所有信息都塞进 context，会让一次性工作集承担数据库、文件系统、日志系统和记忆系统的责任。更稳的 agent 会保留引用，然后在需要时用工具刷新。

8 Compression 和 Compaction

长期任务里，上下文会自然膨胀：工具输出、失败日志、探索路径、用户修正、旧 diff、重复命令都会堆积。Compression 的问题是：什么可以压缩，什么必须原样保留？

可以压缩的是低风险、可重建、已被证实不关键的信息。例如：重复搜索结果、已解决的探索路径、很长但只需要结论的文档、旧工具输出摘要。必须原样保留的是精确证据：失败堆栈、测试输出、错误行号、diff hunk、用户给出的硬约束、安全相关指令、命令和 return code。

内容	处理方式	原因
失败栈	保留关键原文 + 文件引用	行号和异常名不能丢
长日志	摘要 + 附原始路径	需要可追溯
旧探索	压缩成 negative finding	避免重复搜索
用户约束	原样保留	不能被 summary 改写
工具列表	保留最小可用集合	减少工具选择歧义

Compaction 不是把历史缩短，而是把历史变成下一步需要的 state。好的 compaction 会区分已知事实、未验证假设、已排除路径、下一步计划和不可违反约束。

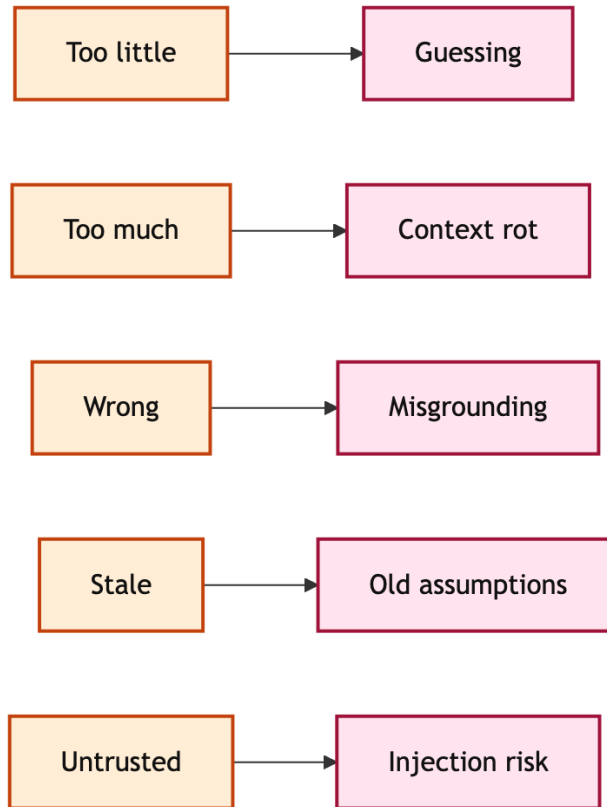
9 Memory Boundary

Memory 很容易被高估。Claude Code docs 明确区分 CLAUDE.md 和 auto memory：前者是用户写的 persistent instructions，后者是 Claude 根据修正和偏好写下的 learnings；两者都作为 context 载入，而不是 enforced configuration。要阻止某个动作，应该用 hooks、permissions、sandbox 或 review gate。

这给 coding agent 一个重要边界：memory 可以减少重复解释，但不能承担 policy。比如“不要改 generated files”如果只写在 memory 里，模型可能忘记或误用；如果这是硬约束，就应该进入 ignore pattern、permission、hook 或 review 检查。

机制	用途	不适合承担
CLAUDE.md AGENTS.md	/ 团队共享规则、运行方式	动态错误日志
Auto memory Session summary	偏好、反复纠正、经验 长任务续接	安全政策 精确证据
Hook / policy Review	强制阻止或检查动作 最终验收和责任归属	背景说明 自动记忆替代

10 Context Failure Modes



第一类失败是 too little context。agent 缺少项目规则、测试命令或相关文件，只能根据通用经验猜。第二类是 too much context。大量无关内容稀释信号，模型无法稳定利用中间信息。第三类是 wrong context。检索到了相似但无关的文件，或者使用了另一个版本的文档。第四类是 stale context。记忆里保存了过期构建命令或旧架构假设。第五类是 untrusted context。issue、网页、依赖 README 里可能包含 prompt injection 或不可信指令。

失败	表现	修复方式
Too little	agent 编造项目事实	先探索、再修改
Too much	关键 evidence 被淹没	选择、排序、压缩
Wrong	改了相似但无关模块	增加路径和调用链证据
Stale	使用旧命令或旧假设	memory audit 和更新时间
Untrusted	遵循外部恶意指令	source trust 和 policy gate

11 Coding-Agent Checklist

开始任务前，可以做一次 context triage。它不是完整计划，而是把“当前最缺的上下文”找出来。很多失败任务并不是模型不会，而是模型在缺少关键上下文的情况下太早开始写 patch。

问题	如果答案不清楚	下一步
怎么算完成?	目标会漂移	要求用户或 issue 给验收标准
哪里失败?	会凭感觉修	先找日志、测试、复现步骤
哪些文件相关?	会全 repo 搜索	先建候选路径和理由
有什么硬约束?	会越界修改	读取 context file 和 policy
怎样验证?	patch 无法信任	先找到测试命令和 oracle

写 context file 时，先从高频失败开始。agent 经常跑错测试，就写测试命令；经常改错目录，就写目录边界；经常忘记 formatter，就写格式化步骤；经常误用生成文件，就写禁止范围。不要一开始写一份面面俱到的长宪法。

做 retrieval 时，要求 agent 先说明它要找什么证据，再读取文件。比如“先列出鉴权相关文件和理由，再读取最可能的两个入口”，比“阅读整个鉴权模块”更好。

做 compression 时，要求 agent 把 summary 标成 facts、hypotheses、excluded paths、next actions。这样下一轮不会把未验证假设当成事实。

做 memory audit 时，定期删除一次性调试结论，保留稳定规则和高频偏好。Memory 应该像一个维护过的 notebook，不应该像一堆聊天残渣。

最小可用 context discipline:

规则写短；证据可追溯；检索按问题进行；summary 区分事实和假设；memory 可审计；安全约束进 policy 而不是只进 prompt。

12 Context QA 回放

一个 context 策略是否有效，最直接的检查方式是 replay。选取一次失败任务，回放 agent 在每一步看到的上下文：它最初知道什么，检索了什么，summary 保留了什么，哪些证据进入了最终 patch，哪些用户约束被遗忘。这个过程能把“模型没做好”拆成上下文链路上的具体问题。

回放点	检查问题	常见修复
Initial prompt	目标和约束是否清楚	改 context file 或任务模板
Retrieval	是否先找证据再下结论	增加 repo map / grep step
Selection	是否把无关内容放进窗口	调整 ranking 和 top-k
Compaction	是否丢掉关键行号或失败栈	facts / hypotheses 分层 summary
Memory	是否用了过时经验	memory audit 和版本标注
Final patch	是否能追溯到证据链	要求 diff rationale 和 test log

Context QA 的目标不是追责，而是找到最便宜的系统修复。若问题出在 initial prompt，就不该换模型；若问题出在 retrieval，就不该只调 summary；若问题出在 memory drift，就不该继续增加长期记忆。好的 replay 会给出一个明确 action：删一条旧 memory、补一条项目规则、改一个检索入口、保留一段原始日志，或给某类任务加验证步骤。

13 Context 更新生命周期

Context 不是一次写完的 prompt，而是一组会老化的工程资产。项目规则、测试命令、目录结构、依赖版本、团队偏好都会变。好的 context engineering 需要 lifecycle：创建、使用、审计、更新、删除。

阶段	应该做什么	坏味道
Create	只写稳定规则和高频流程	把一次性调试结论写进长期文件
Use	agent 引用规则时保留来源	不知道某条规则从哪里来
Audit	定期检查是否过期或冲突	旧命令反复误导任务
Update	随代码/流程变化同步修改	只在聊天里口头纠正
Delete	删除低频、过时、错误记忆	context 文件越来越长

最实用的做法，是把 context 更新和工程事件绑定：测试命令变化时更新 AGENTS.md 或 CLAUDE.md；目录重构后更新 repo guide；一次失败任务暴露了高频错误，就把稳定修正写入 context；如果只是一次性探索结论，则只放在 session summary，不进入长期 memory。这样 context 才像维护过的代码，而不是越积越厚的聊天历史。

14 怎样评估 Context Engineering

Context engineering 的评估不能只看“模型答得对不对”。因为一个上下文策略可能让模型短期通过任务，却长期积累错误记忆；也可能提高成功率，但需要大量手工维护。更好的评估应同时看任务效果、上下文成本和可维护性。

维度	应该看什么	失败信号
Relevance	进入上下文的信息是否支持当前决策	大量无关文件和旧聊天
Grounding	关键结论是否有文件、日志、测试支撑	summary 无来源
Freshness	规则和记忆是否与当前代码一致	旧命令反复出现
Compactness	是否能用更少 token 保留同等信号	上下文越来越长
Maintainability	context files 是否可 review 和版本管理	个人临时 prompt 到处复制

对 coding agent 来说，最实用的评估问题是：同一个任务，如果只给用户 prompt，agent 会不会猜？加上 AGENTS.md 后是否减少重复错误？引入检索后是否更快找到相关文件？压缩后是否仍保留关键证据？memory 更新后是否减少下一次解释成本？这些问题比抽象地问“上下文够不够长”更有意义。

15 局限与开放问题

第一，context freshness 很难。文档、依赖、测试命令、项目架构都会变。Context files 必须跟代码一起演化，否则越“持久”越危险。

第二，context poisoning 是长期风险。agent 从 issue、网页、PR comment 或依赖文档读取内容时，可能读到不可信指令。检索系统不仅要问“相关吗”，还要问“可信吗”。

第三，压缩会带来不可逆损失。summary 可以节省 token，但可能丢掉精确行号、反例和边界条件。关键 evidence 必须保留原文或路径。

第四，不同模型的 context use 能力不同。Lost in the Middle 是重要警示，但不能替代对当前模型、当前任务、当前上下文策略的实际 eval。

16 最终 Takeaway

Context engineering 的核心，是连续地管理模型看到的信息状态。对 coding agent 来说，它比 prompt 更大，比 RAG 更宽，比 memory 更严格：它包括规则、任务、证据、工具、历史、压缩、检索、顺序、信任和更新。

一个好的 coding agent 不需要把整个世界塞进上下文。它需要知道什么应该立刻看，什么应该按需取，什么应该压缩，什么必须原样保留，什么不能信，什么应该写入长期记忆，什么必须交给 policy 和 review。

参考资料

- [1] [Effective Context Engineering](#). finite context, informative-but-tight guidance, just-in-time retrieval, and context curation
- [2] [Context Engineering for AI Agents in OSS](#). AI context files, AGENTS.md adoption, and context-file style variation
- [3] [Lost in the Middle](#). position sensitivity and long-context retrieval caution
- [4] [Retrieval-Augmented Generation](#). parametric and non-parametric memory for retrieval-augmented generation
- [5] [Codex AGENTS.md Guide](#). project guidance, scoped instructions, and concise practical repo context
- [6] [Claude Code Memory](#). CLAUDE.md, auto memory, and context-not-policy distinction
- [7] [Claude Code Workflows](#). codebase exploration, relevant-code finding, and keeping main context clean